



Network Programmability and Automation



Foreword

- New protocols, technologies, and delivery and O&M modes are emerging in the network engineering field. Conventional networks face challenges from new connection requirements, such as requirements for cloud computing and artificial intelligence (AI). Enterprises are also pursuing service agility, flexibility, and elasticity. Against this backdrop, network automation becomes increasingly important.
- Network programmability and automation is to simplify network configuration, management, monitoring, and operations for engineers and improve deployment and O&M efficiency. This course is to help network engineers understand Python programming and implement network automation.



Objectives

- On completion of this course, you will be able to:
 - Describe the difficulties of conventional network O&M.
 - Understand the implementation of network automation.
 - Understand the classification of programming languages.
 - Describe the Python code style.
 - Describe the basic usage of Python telnetlib.



Contents

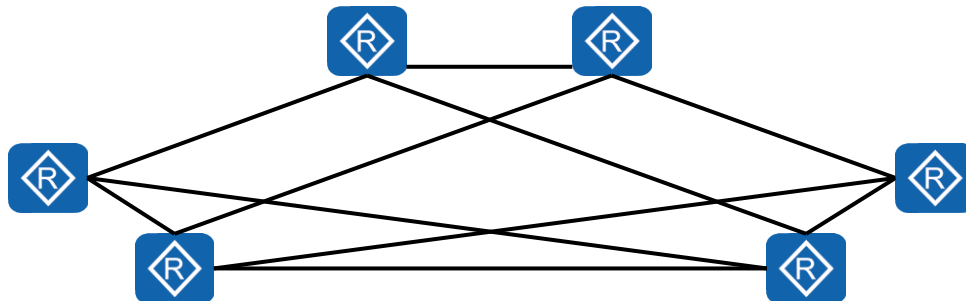
- 1. Introduction to Network Programmability and Automation**
2. Overview of Programming Language and Python
3. Cases



Background: Difficulties in Conventional Network O&M

- Conventional network O&M requires network engineers to manually log in to network devices, query and execute configuration commands, and filter command output. This highly human-dependent working mode is time-consuming, inefficient, and difficult to audit.

Numerous devices
Complex operations
Low efficiency



Network device

Typical O&M Scenarios

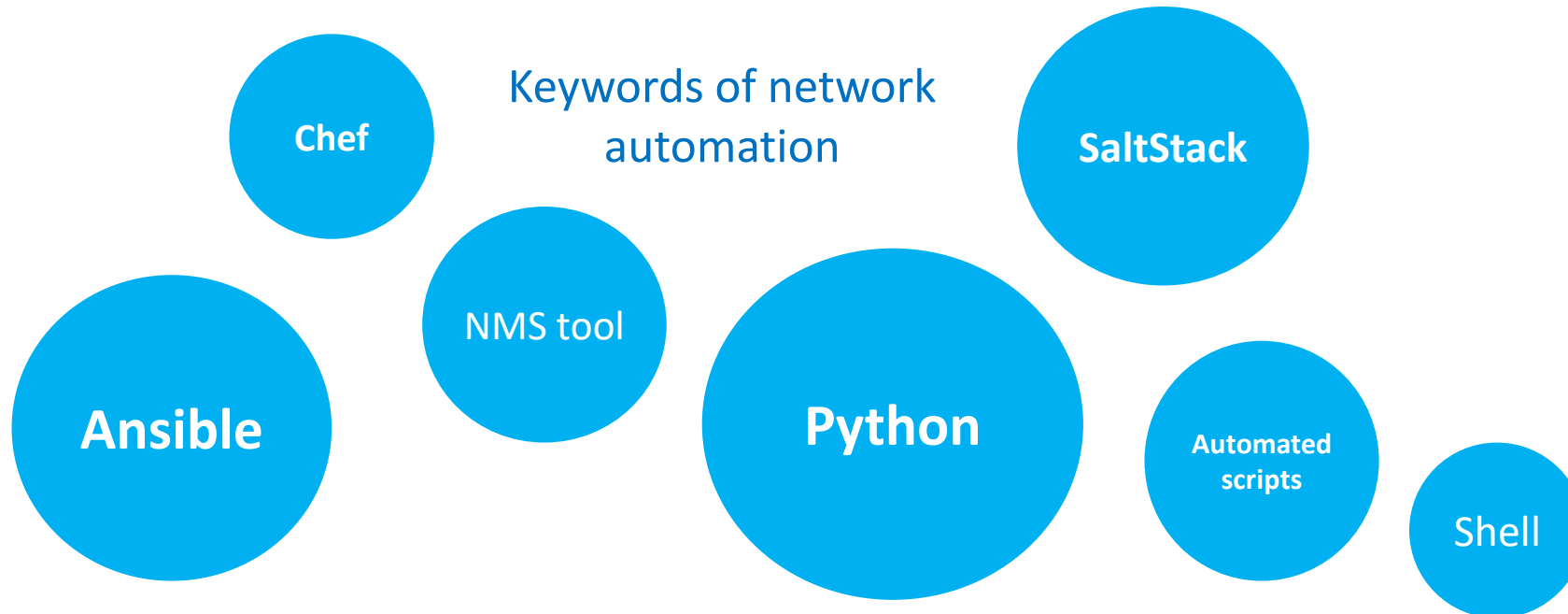
Are the following working scenes familiar to you?

1. Device upgrade: Thousands of network devices reside on a live network. You have to periodically upgrade the devices in batches.
2. Configuration audit: An enterprise needs to audit the configuration of devices every year. For example, the enterprise requires that STelnet be enabled on all devices and spanning tree security be configured on Ethernet switches. In this case, you have to quickly find out the devices that do not meet the requirements.
3. Configuration change: Due to network security requirements, device accounts and passwords need to be changed every three months. You have to delete the original account and create an account on thousands of network devices.



Network Automation

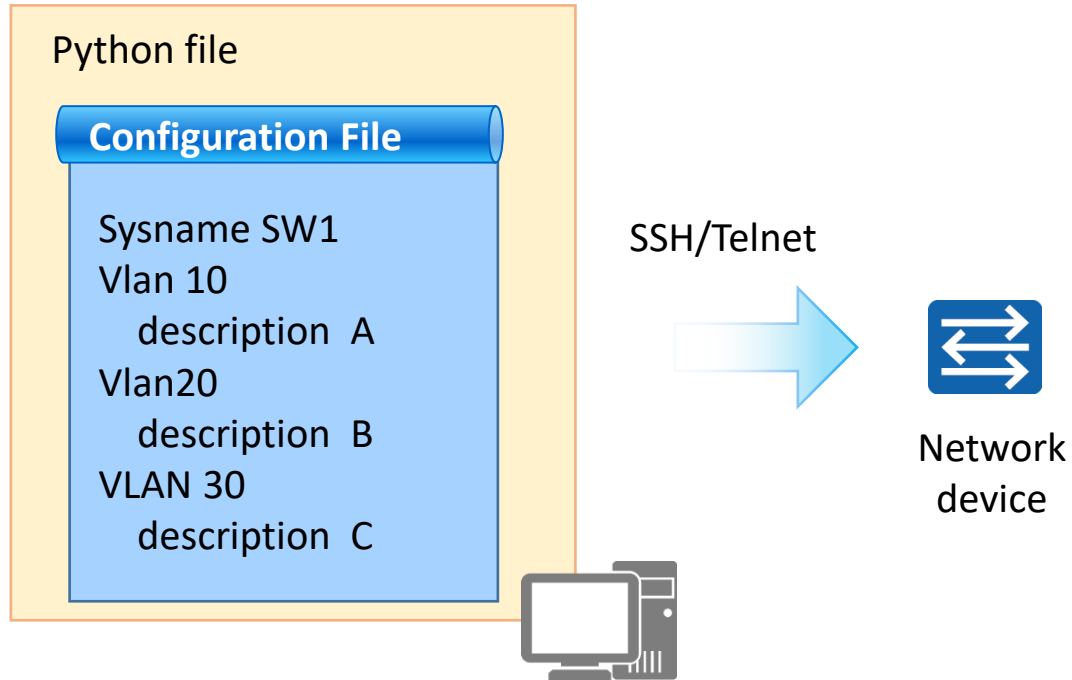
- Network automation: Tools are used to implement automated network deployment, operations, and O&M, gradually reducing dependency on human. This solves the conventional network O&M problems.
- Many open-source tools, such as Ansible, SaltStack, Puppet, and Chef, are available for network automation in the industry. From the perspective of network engineering capability construction, it is recommended that engineers acquire the code programming capability.





Programming-based Network Automation

- In recent years, with the emergence of network automation technologies, Python-based programming capabilities have become a new skill requirement for network engineers.
- Automation script written in Python can execute repeated, time-consuming, and rule-based operations.



Example: Implementing automated device configuration using Python

- What can network automation do? The most intuitive example of network automation is automated device configuration. This process can be divided into two steps: writing a configuration file, and writing Python code to push the configuration file to a device.
- Write the configuration script in command line interface (CLI) mode, and then upload the script to the device using Telnet/SSH. This method is easy to understand for network engineers who are beginning to learn network programmability and automation. This presentation describes how to implement network automation.



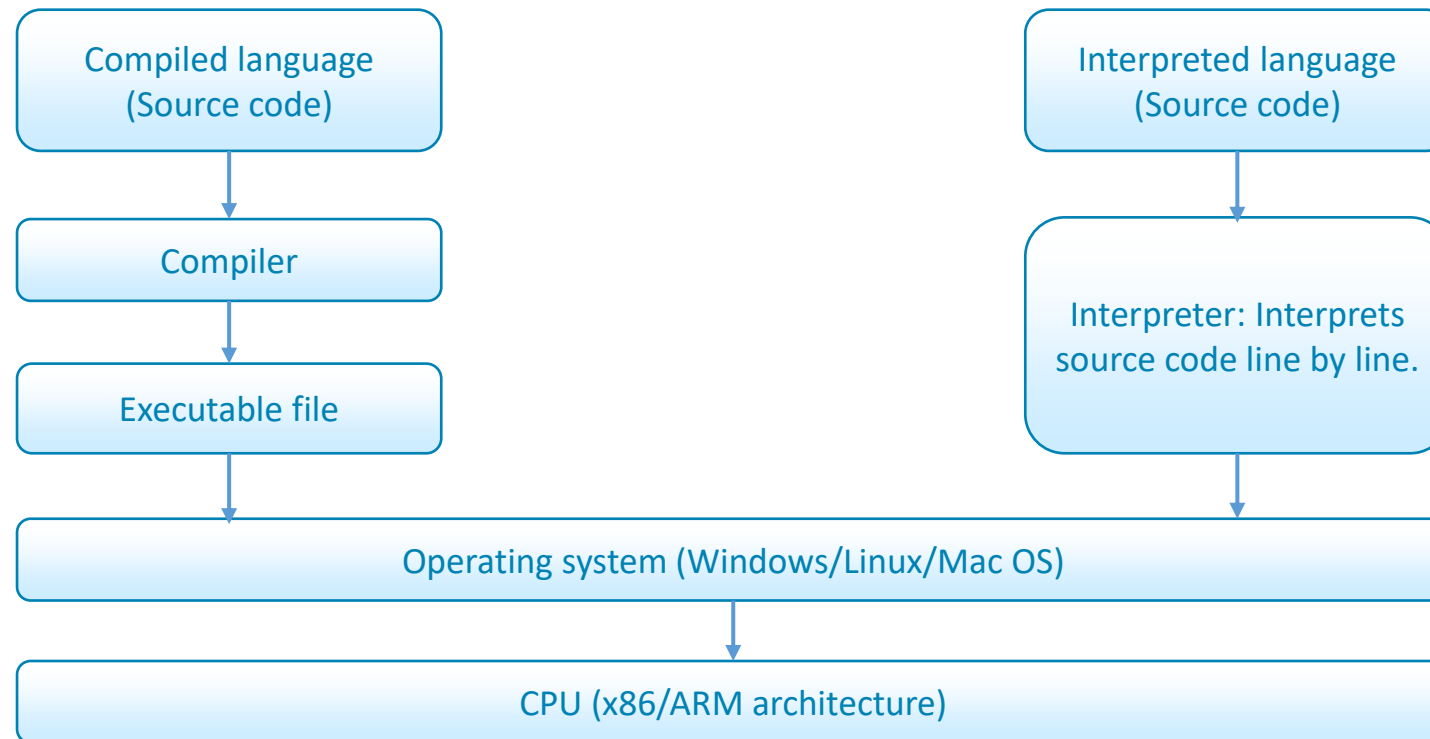
Contents

1. Introduction to Network Programmability and Automation
- 2. Overview of Programming Language and Python**
3. Cases



Programming Languages

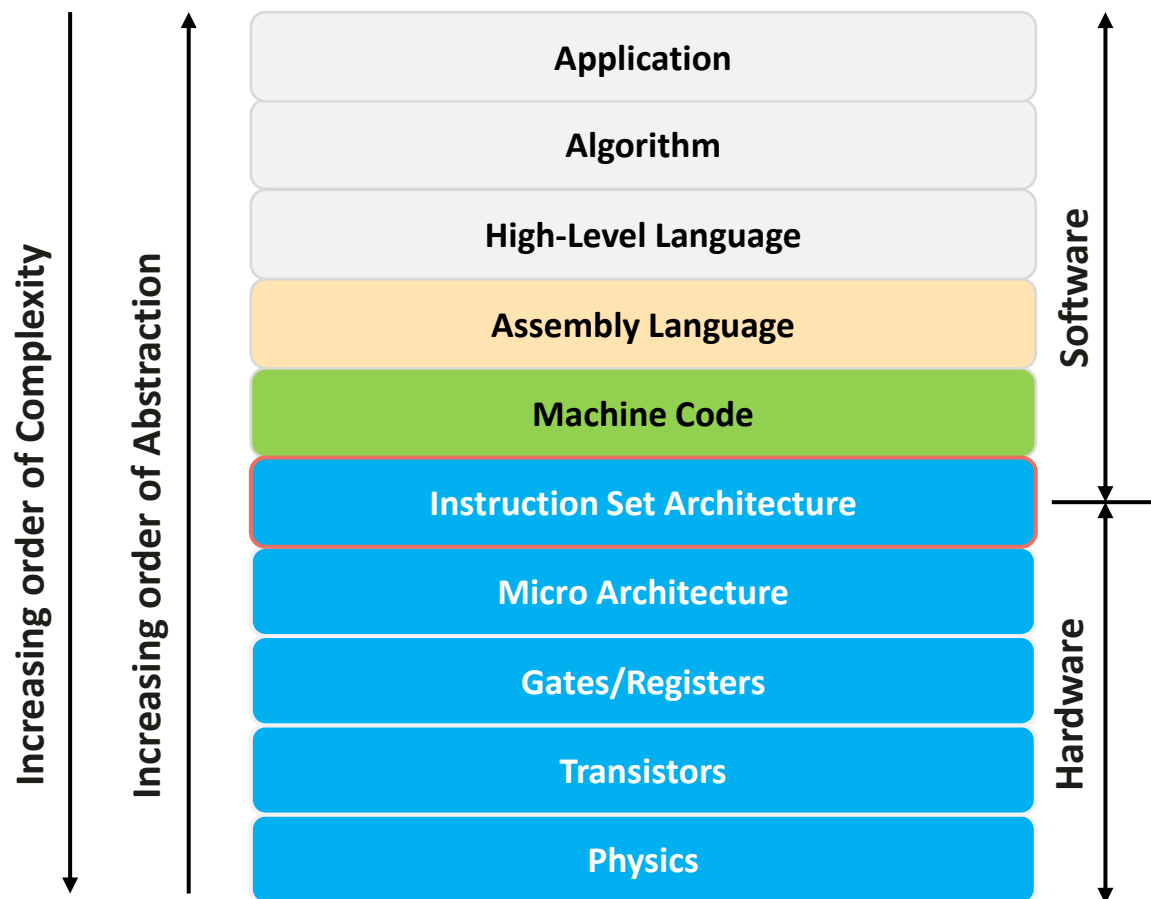
- A programming language is used to write a computer program and control behavior of a computer.
- According to whether compilation is required before execution of a language, the programming language may be classified into the compiled language, and interpreted language that does not need to be compiled.



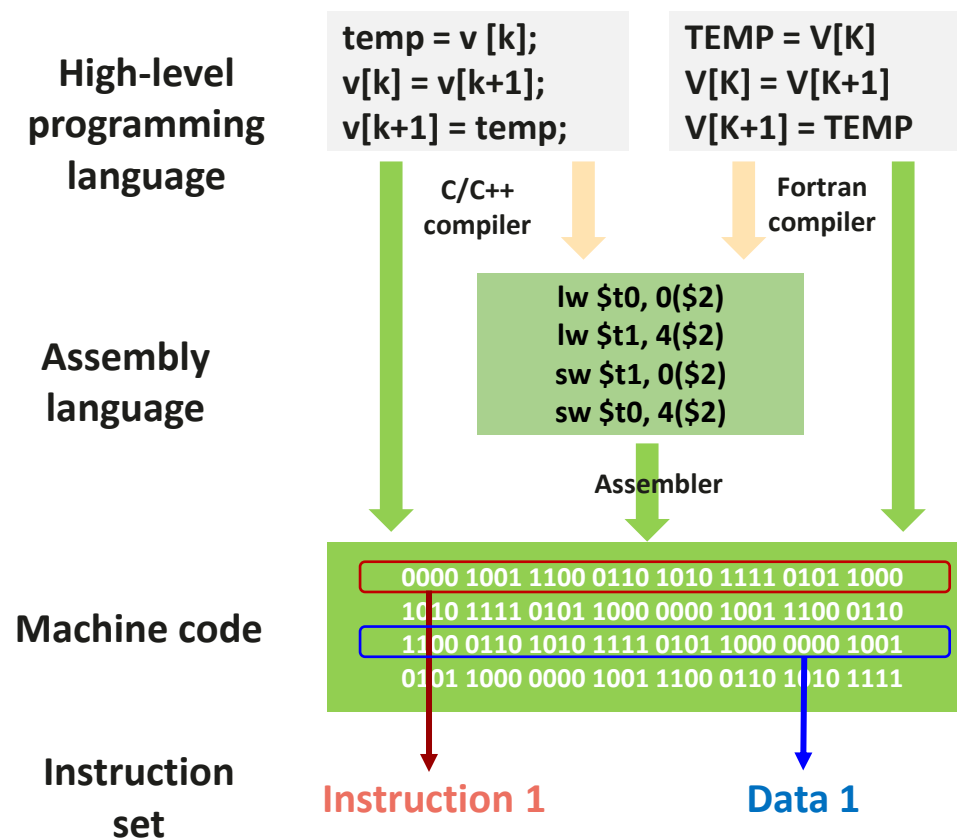


Computing Technology Stack and Program Execution Process

Computing Technology Stack



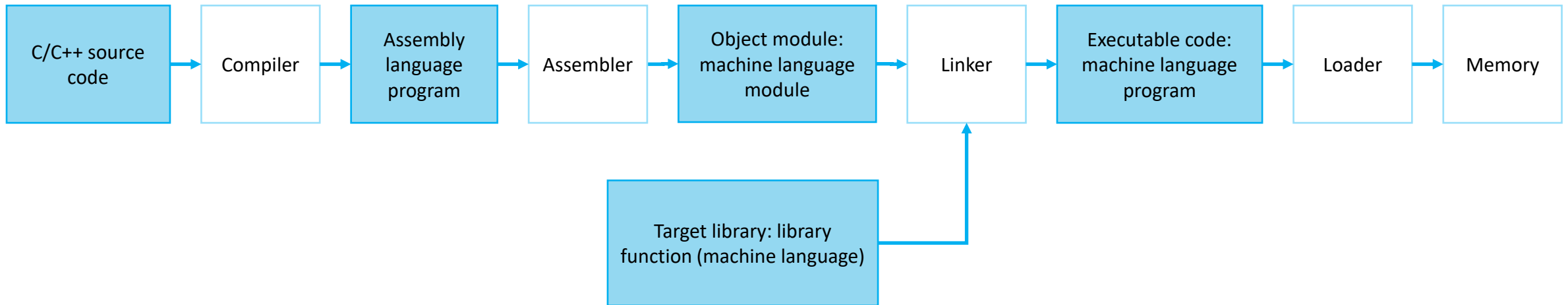
Program Execution Process





High-level Programming Language - Compiled Language

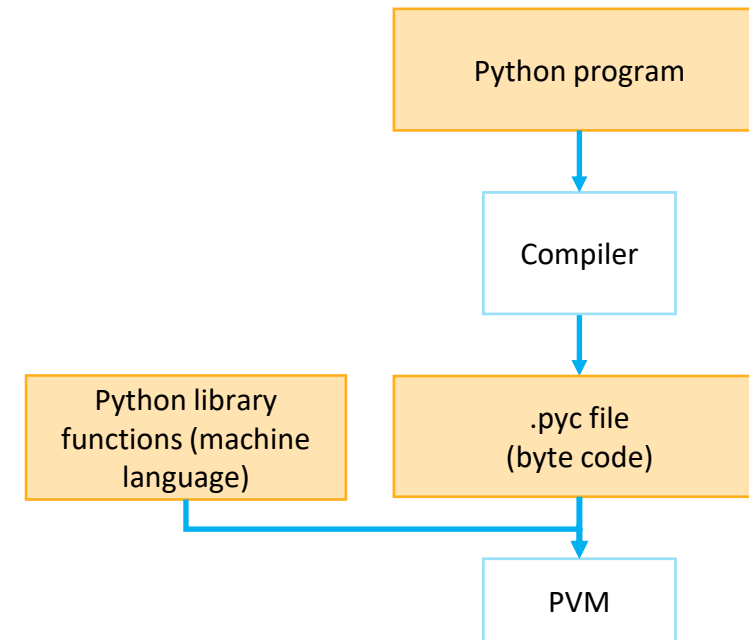
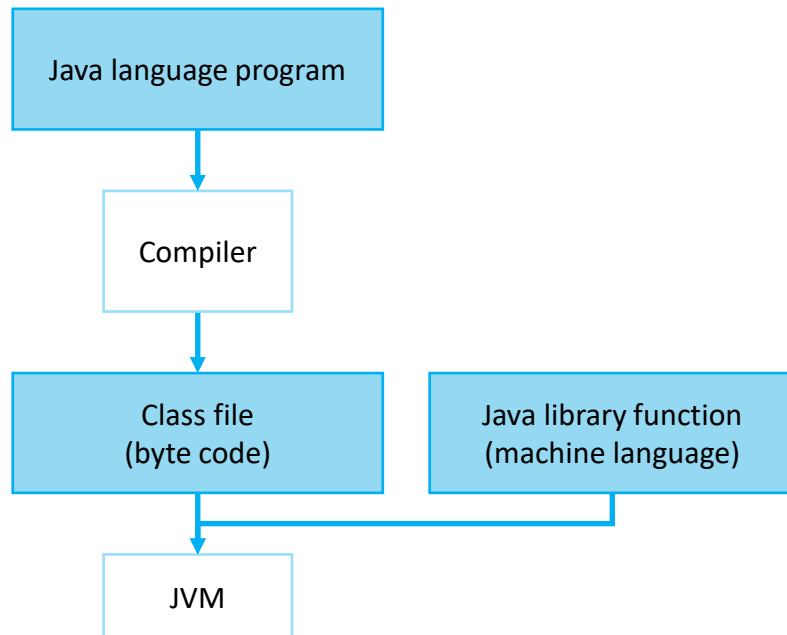
- Compiled language: Before a program in a compiled language is executed, a compilation process is performed to compile the program into a machine language file. The compilation result can be directly used without re-translation during running. Typical compiled languages include C/C++ and Go.
- From source code to program: The source code needs to be translated into machine instructions by the compiler and assembler, and then the linker uses the link library function to generate the machine language program. The machine language must match the instruction set of the CPU, which is loaded to the memory by the loader during running and executed by the CPU.





High-level Programming Language - Interpreted Language

- Interpreted language: Interpreted language programs do not need to be compiled before running. They are translated line by line when running. Typically, Java and Python are interpreted languages.
- Process from source code to programs: Source code of an interpreted language is generated by the compiler and then interpreted and executed by a virtual machine (VM) (for example, JVM/PVM). The VM shields the differences between CPU instruction sets. Therefore, portability of the interpreted language is relatively good.





What Is Python?

- Python is a fully-open-source high-level programming language. Its author is Guido Van Rossum.

Advantages of Python:

- Is a dynamically typed interpreted language with elegant syntax. It allows learners to focus on program logic rather than syntax detail learning.
- Supports both process- and object-oriented programming.
- Provides abundant third-party libraries.
- Is nicknamed the glue language because it can call code written in other languages.

Disadvantages of Python:

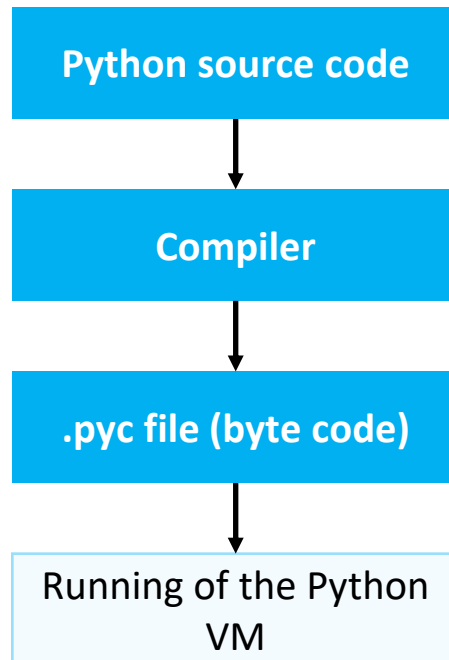
- Runs slow. Is an interpreted language that runs without being compiled. Code is translated line by line at run time into machine code that the CPU can understand, which is time-consuming.

With support for abundant third-party libraries and advantages of the Python language, Python can be used in many fields, such as AI, data science, apps, and scripts for automated O&M.



Python Code Execution Process

Process of compiling and running a Python program



Operation

1. Install Python and the running environment in an operating system.
2. Compile Python source code.
3. The compiler runs the Python source code and generates a .pyc file (byte code).
4. A Python VM converts the byte code into the machine language.
5. Hardware executes the machine language.



Getting Started with Python Code - Interactive Running

- Python runs in either interactive or script mode.
- Interactive programming does not require script files to be created, and code is written in the interactive mode of the Python interpreter.

1. Input --
2. Output --
3. Input --
4. Input --
5. Input --
6. Output --

```
C:\Users\Richard>python
Python 3.7.4 (default, Aug 9 2019, 18:34:13) [MSC v.1915 64 bit (AMD64)] :: Anaconda, Inc.
on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> print ("hello world")
hello world
>>> a = 1
>>> b = 2
>>> print ( a + b )
3
>>>
```



Getting Started with Python Code - Script-based Running

- Code in script mode can run on various Python compilers or in integrated development environments. For example, IDLE, Atom, Visual Studio, Pycharm, and Anaconda provided by Python can be used.

demo.py

```
print ("hello world")  
a = 1  
b = 2  
print ( a + b )
```

1 Write a Python script file (.py).

1. Input --
2. Output --
3. Output --

```
C:\Users\Richard>python demo.py  
hello world  
3
```

2 Execute the script file.



Code Style Guide for Python

- Code style rules refer to naming rules, code indentation, and code and statement segmentation modes that must be complied with when Python is used to write code. Good style rules help improve code readability and facilitate code maintenance and modification.
- For example, the following rules for using semicolons, parentheses, blank lines, and spaces are recommended:

Semicolon

- A semicolon can be added at the end of a line in Python, but is not recommended to separate statements.
- It is recommended that each sentence be in a separate line.

Blank line

- Different functions or statement blocks can be separated by spaces. A blank line helps differentiate two segments of code, improving code readability.

Parentheses

- Parentheses can be used for the continuation of long statements. Avoid unnecessary parentheses.

Space

- Spaces are not recommended in parentheses.
- You can determine whether to add spaces on both ends of an operator.



Code Style Guide for Python - Identifier Naming

- A Python identifier represents the name of a constant, variable, function, or another object.
- An identifier is usually composed of letters, digits, and underscores, but cannot start with a digit. Identifiers are case sensitive and must be unique. If an identifier does not comply with the rules, the compiler will output a `SyntaxError` message when running the code.

1. Assign a value

--

```
User_ID = 10
```

2. Assign a value

--

```
user_id = 20
```

3. Assign a string

--

```
User_Name = 'Richard'
```

4. Assign a value

--

```
Count = 1 + 1
```

5. Incorrect identifier

--

```
4_passwd = "Huawei"
```

```
print ( User_ID )
```

```
print ( user_id )
```

```
print ( User_Name )
```

```
print ( Count )
```

```
print ( 4_passwd )
```

`print()` is a built-in function of Python and is used to output content in parentheses.

Question: What is the output of the print command on the right?



Code Style Guide for Python - Code Indentation

- In Python programs, code indentation represents the scope of a code block. If a code block contains two or more statements, the statements must have the same indentation. For Python, code indentation is a syntax rule that uses code indentation and colons to distinguish between layers of code.
- When writing code, you are advised to use four spaces for indentation. If incorrect indentation is used in the program code, an IndentationError error message is displayed during code running.

Correct indentation --

Correct indentation --

Incorrect indentation --

```
if True:
    print ("Hello")
else:
    print (0)

a = "Python"
    print (a)
```



Code Style Guide for Python - Using Comments

- Comments are explanations added to programs to improve program readability. In the Python program, comments are classified into single-line comments and multi-line comments.
- A single-line comment starts with a pound sign (#).
- A multi-line comment can contain multiple lines, which are enclosed in a pair of three quotation marks ('''...''' or """ ... """).

Single-line comment --

```
# Assign a string to a.  
a = "Python"  
print (a)
```

Multi-line comment --

```
"""  
The output is Python.  
"""
```



Code Style Guide for Python - Source Code File Structure

- A complete Python source code file generally consists of interpreter and encoding format declaration, document string, module import, and running code.
- If you need to call a class of a standard library or a third-party library in a program, use "import" or "from... import" statement to import related modules. The import statement is always after the module comment or document string (docstring) at the top of the file.

Interpreter declaration	--	<code>#!/usr/bin/env python</code>
Encoding format declaration	--	<code>#-*- coding:utf-8 -*-</code>
Module comment or document string	--	Description of a document (docstring) This document is intended for... <code>"""</code>
Time when a module is imported	--	<code>import time</code>
Code is running	--	<code>...</code>



Python Functions and Modules

- A function is a block of organized, reusable code that is used to perform a single, related action. It can improve the modularity of the program and code utilization. The function is defined using the `def` keyword.
- A module is a saved Python file. Modules can contain definitions of functions, classes, and variables that can then be utilized in other Python programs. The only difference between a module and a regular Python program is that the module is used for importing by other programs. Therefore, a module usually does not have a main function.

`demo.py`

```
def sit(): #Define a function.  
    print ('A dog is now sitting')  
  
sit() #Call a function.
```

Execution result:

A dog is now sitting.

1 Write a Python file (.py).

`test.py`

```
import demo #Import a module.  
  
demo.sit() #Call a function.
```

Execution result:

A dog is now sitting.
A dog is now sitting.

2 Import a module.



Python Classes and Methods

- A class is a collection of properties and methods that are the same. The class keyword is used to define a class.
- The function of an instantiated class is called a method. When you define a method, a class must carry the **self** keyword, which indicates the instance of the class.

demo.py

```
class Dog(): #Define a class.  
    def sit(self): #Define a method.  
        print("A dog is now sitting.")  
  
Richard = Dog() #The class is instantiated.  
print (type(Richard.sit)) #The function of an instantiated type is called a  
method.  
print (type(Dog.sit)) #The type is function.
```

Execution result:

```
<class 'method'>  
<class 'function'>
```

1 Write a Python file (.py).

test.py

```
import demo  
  
demo.Dog.sit
```

Execution result:

```
A dog is now sitting.  
<class 'method'>  
<class 'function'>
```

2 Import a module.



Introduction to telnetlib

- telnetlib is a module in the standard Python library. It provides the telnetlib.Telnet class for implementing the Telnet function.
- Different methods in the telnetlib.Telnet class are called to implement different functions.

Import the Telnet class of the telnetlib module. --
Create a Telnet connection to a specified server. --
Invoke the read_all() method. --

```
from telnetlib import Telnet
tn = Telnet(host=None, port=0[, timeout])
tn.read_all()
...
```

Method	Function
Telnet.read_until (expected, timeout=None)	Read data until a given byte string, <i>expected</i> , is encountered or until <i>timeout</i> seconds have passed.
Telnet.read_all ()	Read all data until EOF as bytes; block until connection closed.
Telnet.read_very_eager()	Read everything that can be without blocking in I/O (eager). Raise EOFError if connection closed and no cooked data available. Return b" if no cooked data available otherwise. Do not block unless in the midst of an IAC sequence.
Telnet.write(buffer)	Write a byte string to the socket, doubling any IAC characters.
Telnet.close()	Close the connection.



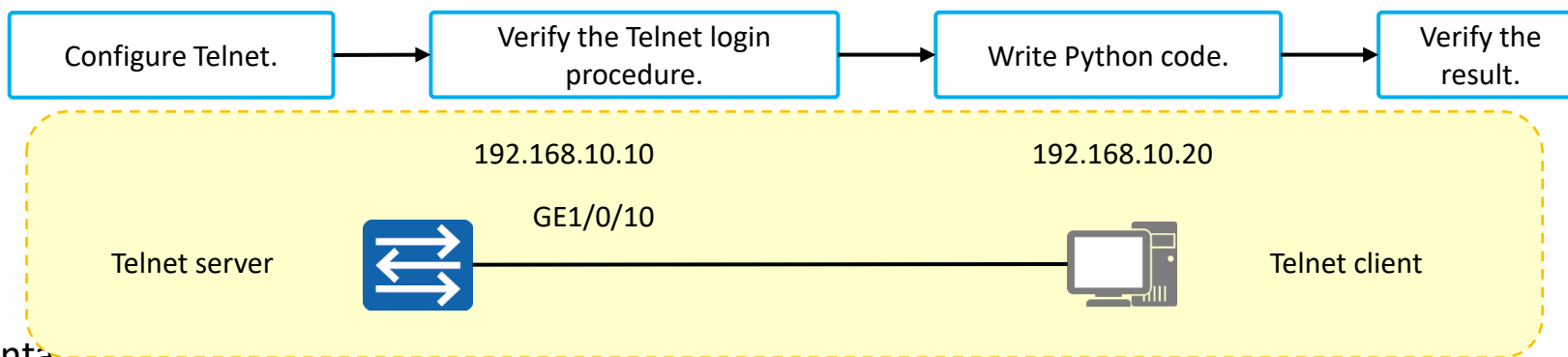
Contents

1. Introduction to Network Programmability and Automation
2. Overview of Programming Language and Python
- 3. Cases**



Case: Logging In to a Device Using telnetlib

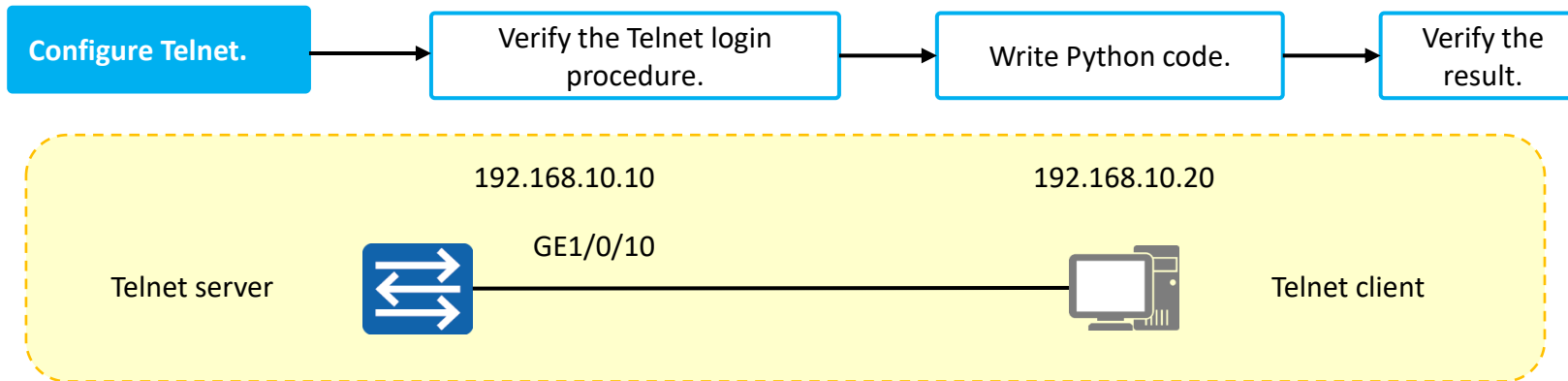
- Case description :
- A network device functions as a Telnet server, and the Python telnetlib needs to be used as a Telnet client to log in to the device.



- The implementation process is as follows.
- Configure the Telnet service.
- Manually verify and view the Telnet login procedure as a reference for code implementation.
- Compile and run Python code.
- Verify the result.



Case: Logging In to a Device Using telnetlib



Configure the IP address of interface on the device:

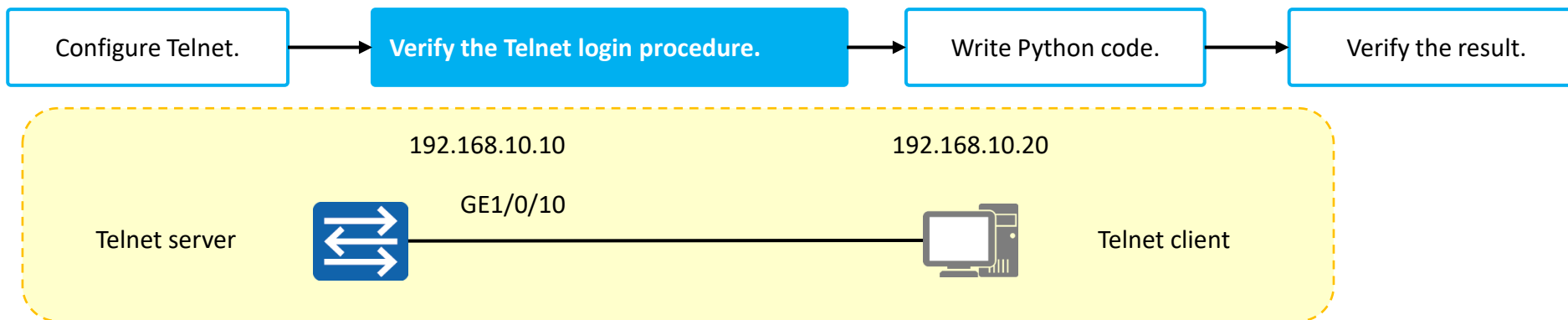
```
[Huawei] interface GE 1/0/0
[Huawei -GE1/0/0] ip add 192.168.10.10 24
[Huawei -GE1/0/0] quit
```

Configure the Telnet service:

```
[Huawei] user-interface vty 0 4
[Huawei-ui-vty0-4] authentication-mode password
[Huawei-ui-vty0-4] set authentication password simple Huawei@123
[Huawei-ui-vty0-4] protocol inbound telnet
[Huawei-ui-vty0-4] user privilege level 15
[Huawei-ui-vty0-4] quit
[Huawei] telnet server enable
```



Case: Logging In to a Device Using telnetlib



Telnet login:

1 Run a login command.
Command output

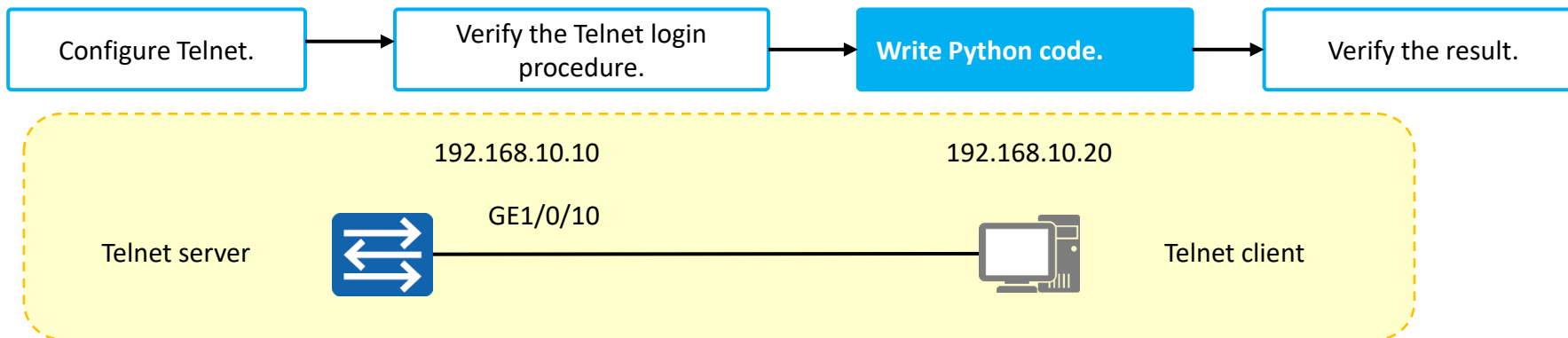
```
C:\Users\Richard>telnet 192.168.10.10
Login authentication
```

2 Enter a password.
Command output

```
Password:
Info: The max number of VTY users is 5, and the number of current VTY users on line is 1.
The current login time is 2020-01-15 21:12:57.
<Huawei>
```



Case: Logging In to a Device Using telnetlib



Imports the module.

Sets the IP address for a host.

Sets the password for logging in to the device.

Logs in to the host through Telnet.

Prints data until **Password:** is displayed.

Sets an ASCII password and starts a new line.

Prints data until **<Huawei>** is displayed.

Closes the Telnet connection.

```
-- import telnetlib
-- host = '192.168.10.10'
-- password = 'Huawei@123'

-- tn = telnetlib.Telnet(host)
-- tn.read_until(b'Password:')
-- tn.write(password.encode('ascii') + b"\n")
-- print (tn.read_until(b'<Huawei>').decode('ascii'))
-- tn.close()
```



Case: Running Result Comparison

Configure Telnet.

Verify the Telnet login
procedure.

Write Python code.

Verify the result.

Manual Telnet login result:

```
C:\Users\Richard>telnet 192.168.10.10
```

```
Login authentication
```

```
Password:
```

```
Info: The max number of VTY users is 5, and the number of current VTY users on line is 1. The current login time is 2020-01-15 21:12:57.
```

```
<Huawei>
```

Python code execution
result:

```
#Run Python code in the compiler.
```

```
Info: The max number of VTY users is 5, and the number  
of current VTY users on line is 1.
```

```
The current login time is 2020-01-15 22:12:57.
```

```
<Huawei>
```



Quiz

1. Python is a compiled language. ()
 - A. True
 - B. False
2. How to create VLAN 10 using telnetlib?



Summary

- Network automation uses tools to implement automated network deployment, operation, and O&M, gradually reducing dependency on people. You can use a programming language or tool to implement the network automation.
- Python is a fully-open-source high-level programming language that is simple syntax and is easy to learn. It has rich standard libraries and third-party libraries, which are applicable to the network engineering field.
- The telnetlib module of Python provides the telnetlib.Telnet class for implementing the Telnet function. It helps you enter the network programmability and automation world!



More Information

- For more information about Python, visit <https://www.python.org/>.

The background of the slide features a blue-tinted image of a modern office interior. In the foreground, several groups of business professionals are silhouetted against the bright light coming from large windows. They appear to be in various stages of collaboration, some looking at documents or devices. The windows in the background offer a view of a dense urban skyline with numerous skyscrapers. The overall atmosphere is professional and forward-looking.

Thank You
www.huawei.com